

1.3. Processos de Software

Disciplina: Engenharia de Software I
Curso: Bacharel em Sistemas de Informação

Professor: M.e Jullian Hermann Creutzberg
2026



O processo de software

- Um conjunto estruturado de atividades necessárias para desenvolver um sistema de software.
- Existem vários processos de desenvolvimento de software diferentes mas todos envolvem:
 - ✓ **especificação** – definição do quê o sistema deve fazer;
 - ✓ **projeto e implementação** – definição da organização do sistema e implementação do sistema;
 - ✓ **validação** – checagem de que o sistema faz o que o cliente deseja;
 - ✓ **evolução** – evolução em resposta a mudanças nas necessidades do cliente.
- Um modelo de processo de desenvolvimento de software é uma representação abstrata de um processo. Ele apresenta uma descrição do processo de uma perspectiva em particular.

Descrições de processo de software

- Quando descrevemos e discutimos processos, geralmente falamos sobre as atividades desses processos, tais como especificação de modelo de dados, desenvolvimento de interface de usuário, etc. e organização dessas atividades.
- Descrições de processos também podem incluir:
 - ✓ **Produtos**, que são os resultados de uma atividade do processo;
 - ✓ **Papéis**, que refletem as responsabilidades das pessoas envolvidas no processo;
 - ✓ **Pré e pós-condições**, que são declarações que são verdadeiras antes e depois de uma atividade do processo ser executada, ou um produto produzido.

Modelos de processo de software

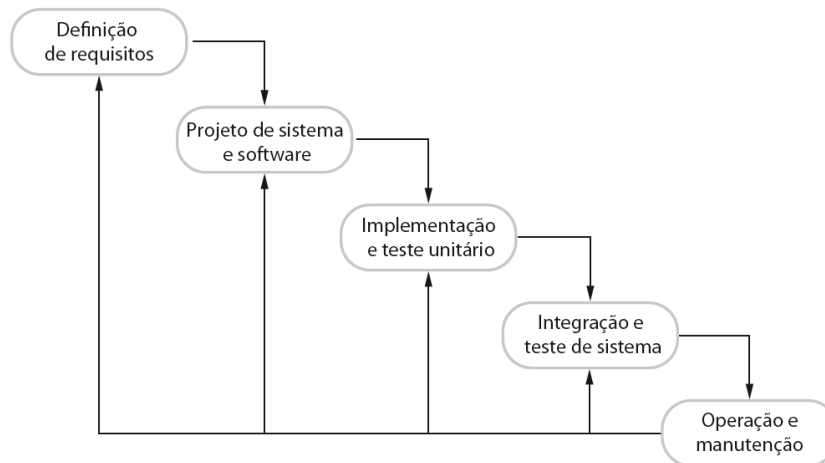
- **Modelo Cascata** – Modelo dirigido a planos. Fases de especificação e desenvolvimento separadas e distintas.
- **Desenvolvimento Incremental** – Especificação, desenvolvimento e validação são intercaladas. Pode ser dirigido a planos ou ágil.
- **Engenharia de software orientada a reuso** – O sistema é montado a partir de componentes já existentes. Pode ser dirigido a planos ou ágil.

Na realidade a maioria dos grandes sistemas são desenvolvidos usando um processo que incorpora elementos de todos esses modelos.

Apresentação Ciclos de Vida: **Atividade 02**

Ciclos de Vida

O modelo cascata



Fases do modelo cascata

- Existem fases identificadas e separadas no modelo cascata:
 - ✓ Análise e definição de requisitos
 - ✓ Projeto de sistema e software
 - ✓ Implementação e teste de unidade
 - ✓ Integração e teste de sistema
 - ✓ Operação e manutenção
- O principal inconveniente do modelo cascata é a dificuldade de acomodação de mudanças depois que o processo já foi iniciado. Em princípio, uma fase precisa ser completada antes de se mover para a próxima fase.

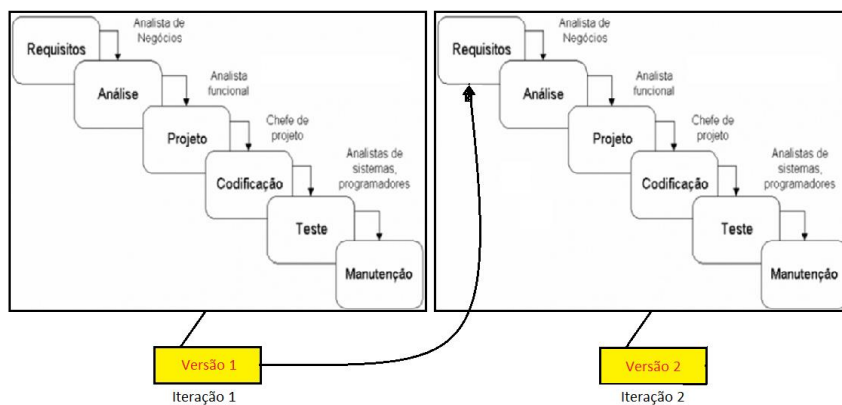
Problemas do modelo cascata

- Divisão inflexível do projeto em estágios distintos torna difícil responder às mudanças nos requisitos do cliente.
 - ✓ Por isso esse modelo só é apropriado quando os requisitos são bem entendidos e as mudanças durante o processo de projeto serão limitadas.
 - ✓ Poucos sistemas de negócio possuem requisitos estáveis.
- O modelo cascata é mais usado em projetos de engenharia de grandes sistemas onde o sistema é desenvolvido em vários locais.
 - ✓ Nessas circunstâncias, a natureza do modelo cascata dirigida a planos ajuda a coordenar o trabalho.

Desenvolvimento iterativo

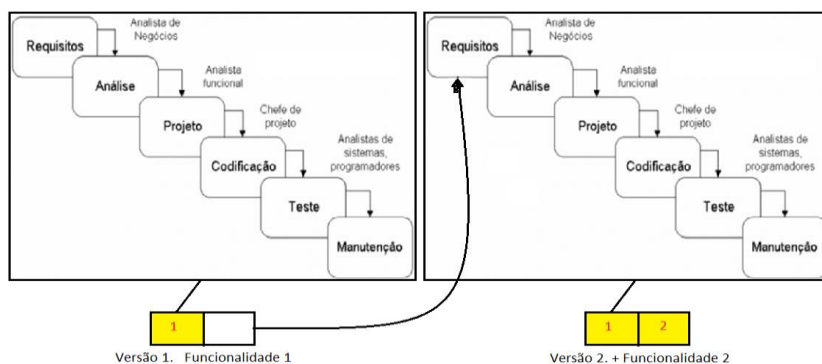
- Liberações intermediárias são feitas.
- Embora a cada liberação um conjunto de requisitos esteja pronto, somente na última liberação é que pode-se utilizar o software completo;
- O modelo puramente iterativo consiste em uma série de iterações do modelo cascata aplicado.

Desenvolvimento iterativo

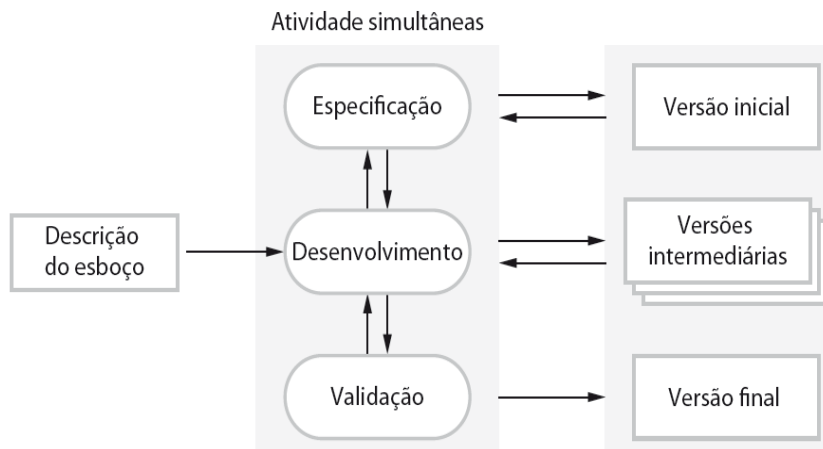


Desenvolvimento incremental

- Liberações mais completas são realizadas.
- A cada liberação o usuário já pode utilizar partes do sistema ou partes de rotinas do sistema.



Desenvolvimento incremental



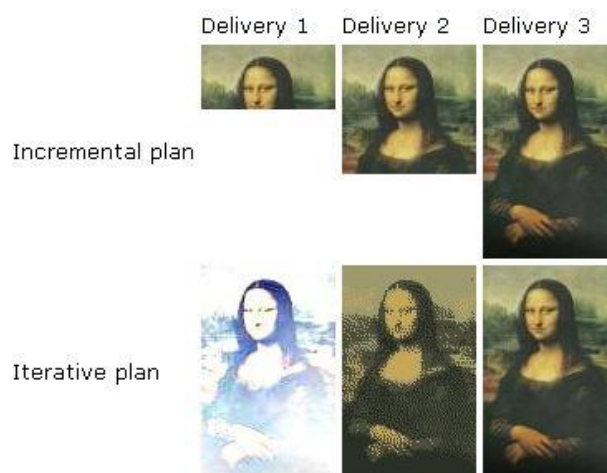
Benefícios do desenvolvimento incremental

- O custo para acomodar mudanças nos requisitos do cliente é reduzido.
 - ✓ A quantidade de análise e documentação que precisa ser feita é bem menor do que a necessária no modelo cascata.
- É mais fácil obter feedback do cliente sobre o trabalho de desenvolvimento que tem sido feito.
 - ✓ Os clientes podem comentar demonstrações do software e ver quanto foi implementado.
- Possibilidade de mais rapidez na entrega e implantação de software útil para o cliente.
 - ✓ Os clientes podem usar e obter ganhos do software mais cedo do que é possível no processo cascata.

Problemas do desenvolvimento incremental

- O processo não é visível.
 - ✓ Gerentes precisam de entregas regulares para medir o progresso. Se os sistemas são desenvolvidos de forma rápida, não é viável do ponto de vista do custo produzir documentação para refletir todas as versões do sistema.
- A estrutura do sistema tende a degradar conforme novos incrementos são adicionados.
 - ✓ A menos que tempo e dinheiro sejam gastos na reconstrução para melhorar o software, as mudanças regulares tendem a corromper a estrutura do sistema. A incorporação posterior de mudanças no software se torna progressivamente mais difícil e cara.

Incremental x Iterativo



Desenvolvimento iterativo e incremental

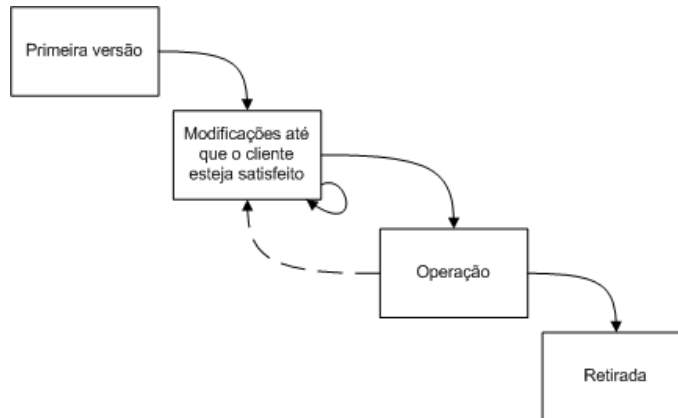
- Combina elementos do modelo iterativo e incremental, entregando partes do desenvolvimento a cada ciclo.



Constrói Concerta

- Basicamente um modelo caótico.
- Baseado no conceito de “operação”.
- Encontrado na maioria das empresas que produzem software.
- O software é constantemente retrabalhado sobre uma mesma versão.
- Da a impressão que não existem modificações que tragam benefícios aos usuários.
- Pode funcionar razoavelmente para micro projetos (<400 horas).

Constrói Concerta



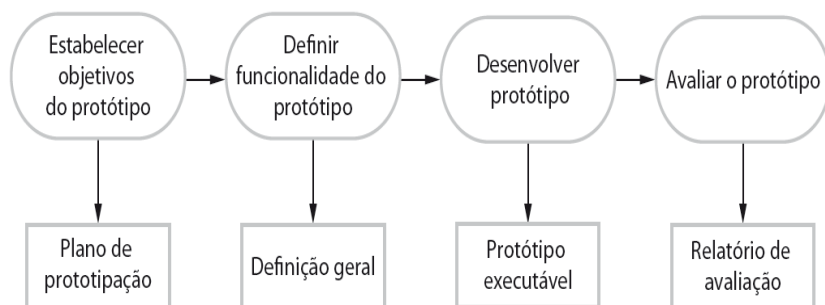
Prototipação

- Um protótipo é uma versão inicial de um sistema usada para demonstrar conceitos e testar opções de projeto.
- Um protótipo pode ser usado:
 - ✓ No processo de engenharia de requisitos para ajudar na elicitação e validação de requisitos;
 - ✓ Nos processos de projeto para explorar opções e desenvolver um projeto de interface de usuário;
 - ✓ No processo de testes para executar testes fim-a-fim.

Benefícios da prototipação

- Melhoria do uso do software.
- Maior proximidade com as necessidades do usuário.
- Melhorias na qualidade do projeto.
- Maior manutenibilidade.
- Reduzir esforços de desenvolvimento.

O processo de desenvolvimento de protótipo



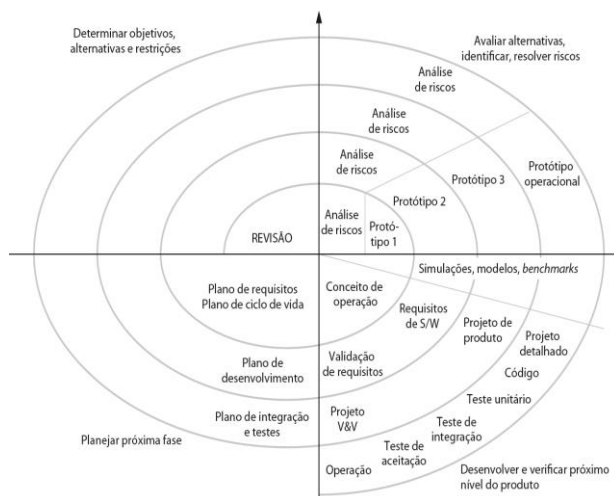
Problemas da prototipação

- O cliente vê o que parece ser uma versão executável do software
- Ainda existe muito por fazer
- O desenvolvedor faz algumas concessões na implementação a fim de conseguir rapidamente um protótipo executável
- Depois para desenvolver o produto final, encontram-se muitos problemas

Modelo espiral de Boehm

- O processo é representado como uma espiral ao invés de uma sequência de atividades com retornos.
- Cada loop na espiral representa uma fase do processo.
- Não existem fases fixas como especificação ou projeto – os loops na espiral são escolhidos de acordo com a necessidade.
- Os riscos são avaliados explicitamente e resolvidos no decorrer do processo.

Modelo espiral de Boehm

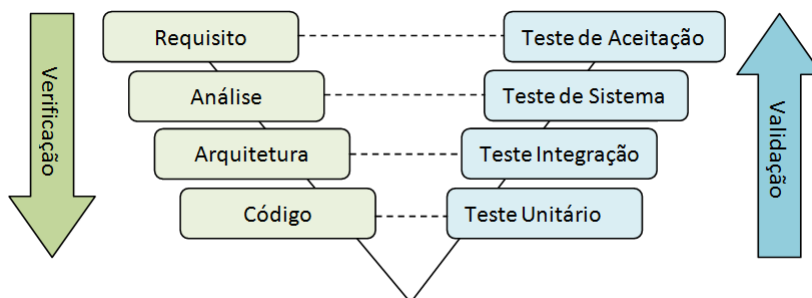


Setores do modelo espiral

- Definição de objetivos
 - ✓ São identificados os objetivos específicos para cada fase.
- Avaliação e redução de riscos
 - ✓ Os riscos são avaliados e atividades executadas para reduzir os principais riscos.
- Desenvolvimento e validação
 - ✓ Um modelo de desenvolvimento para o sistema é escolhido, pode ser qualquer um dos modelos genéricos.
- Planejamento
 - ✓ O projeto é revisto e a próxima fase da espiral é planejada.

Modelo em V

- Evolucionário, parte do princípio de que os procedimentos de verificação de conformidade devem ocorrer durante as fases de concepção e a validação durante o teste.



Atividade 03

• Estudos de Caso - Ciclos de Vida

- A atividade consiste na seguinte dinâmica: de acordo com os ciclos de vida de software apresentados e discutidos, sua equipe de técnicos e analistas deverá escolher um determinado modelo (cascata, espiral, prototipação, iterativo, incremental, iterativo/incremental, ou outro), e aplicar as técnicas do modelo escolhido para o estudo de caso do cliente.
- Deverá apresentar qual modelo se encaixa melhor na produção do software requisitado, especificando as vantagens e desvantagens da escolha do modelo de ciclo de vida de software.
- Deverá utilizar livros ou outras referências de artigos na internet para defender o modelo aplicado em seu estudo de caso.
- Participação: 1,0 Ponto na Nota de Atividades do Semestre
- Tempo para análise: 40 minutos

Modelos de Maturidade

CMMI



CMMI®

- **CMMI**: Trata-se de um **metamodelo** definido por um conjunto de **capacidades** que definem o nível de maturidade no desenvolvimento de software de uma organização;
- Reconhecido internacionalmente por ajudar as organizações a alcançarem processos maduros, estáveis e consistentes;

CMMI

- **CMMI** significa *Capability Maturity Model Integration* – Modelo Integrado de Maturidade e Capacidade, é um modelo alinhado com a norma ISO/IEC 15504
- Dividido em 5 níveis de maturidade que representam um caminho de melhoria de processos para toda a organização

CMMI

- **Nível 1 - Inicial:**
 - Os processos são caóticos;
 - Quando neste nível, padrões não existem, ou se existem, não são seguidos;
 - Problemas no cumprimento de prazos e custos, bem como de requisitos;
 - A organização sobrevive na dependência de talentos individuais;

CMMI

- **Nível 2 - Gerenciado:**

- Os projetos da organização possuem requisitos gerenciados e processos planejados, medidos e controlados;
- A organização consegue cumprir os planos no desenvolvimento dos projetos;
 - *os cronogramas serem periodicamente revistos;*

CMMI

- **Nível 3 – Definido:**

- Os processos são bem caracterizados e entendidos;
- A padronização de processos possibilita maior consistência nos produtos gerados pela organização;
- Na descrição dos processos são usados padrões, procedimentos, ferramentas e métodos bem definidos;

CMMI

- **Nível 4 – Gerenciado Quantitativamente:**

- Utiliza-se métodos estatísticos e outras técnicas quantitativas;
- No nível 4 dados sobre os processos são coletados e analisados estatisticamente para acompanhamento;

CMMI

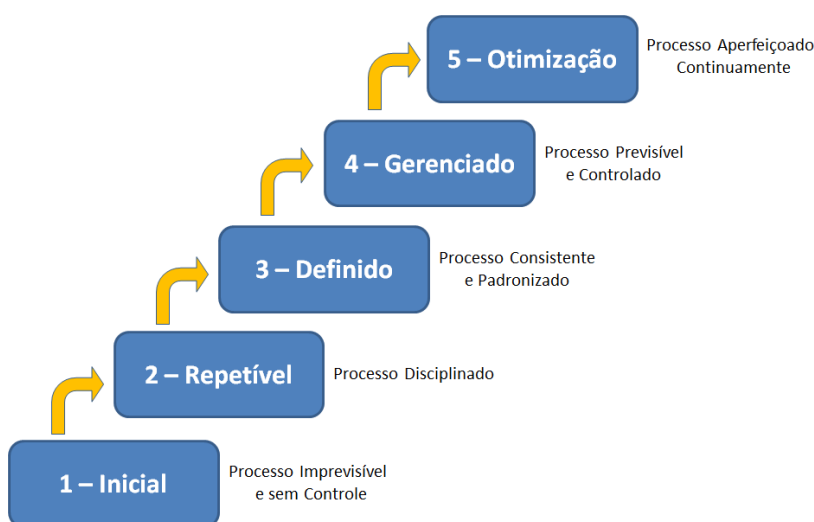
- **Nível 5 – Otimizado:**

- Os processos são continuamente melhorados;
- Esta melhoria é conquistada através de inovações e busca por novas tecnologias;
- A Melhoria de processos é uma tarefa de todos

CMMI

- O modelo em sua versão 2.0, lançada em 2018, preza muito mais a melhoria de processos, pois agora, em todos os níveis a organização deverá apresentar resultados gerados a partir de ações de melhoria.
- O conteúdo do modelo é apresentado de maneira mais amigável e **contempla conceitos ágeis e a metodologia Scrum**, que já era utilizada em diversas empresas certificadas, que tinha um esforço para realizar a equivalência na execução da metodologia com o modelo CMMI.
- Já a versão 3.0, lançada em 2023, tem seu foco principal em segurança, proteção e gestão de dados, reforçando o ciclo de melhoria contínua.

CMMI



MPS.br



- **MPS.br – Melhoria do Processo de Software Brasileiro**
- É um modelo em constante evolução, lançado em 2003;
- Versão mais atual → MPS-SW:2024;
- Nível de maturidade no MPS.br, significa o grau de melhoria de processo;
- Ao todo são 7 níveis.

MPS.br

a) **Em Otimização**

Analisa causa de problemas e resolução

b) **Gerenciado quantitativamente**

Gerência de Projetos (evolução)

c) **Definido**

Análise de Decisão e Resolução
Desenvolvimento para Reutilização
Gerência de Riscos
Gerência de Reutilização (evolução)

d) **Largamente definido**

Desenvolvimento de Requisitos
Projeto e Construção do Produto
Integração do Produto
Verificação / Validação

MPS.br

e) Parcialmente definido

Avaliação e Melhoria do Processo Organizacional
 Definição do Processo Organizacional
 Gerência de Reutilização / Gerência de Recursos Humanos
 Gerência de Projetos (evolução)

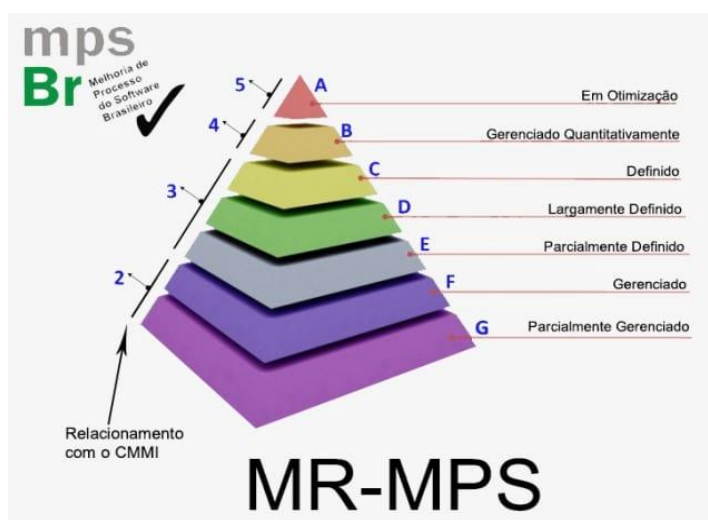
f) Gerenciado

Medição / Gerência de Configuração
 Aquisição / Garantia de Qualidade

g) Parcialmente gerenciado

Gerência de Requisitos
 Gerência de Projetos

MPS.br



Referências

- SOMMERVILLE, I. **Engenharia de software**. 9. ed. São Paulo: Pearson, 2011. (Capítulo 2)
- <https://softex.br/mpsbr/>